Python for Finance

Companion Slides to the Practical Sessions (TD 1–6)

M1 Economie – Finance, Université Paris Dauphine

Matéo Molinaro

2026

How to Use These Slides

These slides are a **companion** to the Jupyter notebooks used in the practical sessions (TD). They give the **full derivations** behind each practical session, self-contained, so that the slide deck can be read on its own. They point back to the notebook exercise that implements each result – but the derivations themselves do not depend on the code.

Companion sessions

TD1: Python foundations, Black–Scholes Greeks, Monte Carlo • TD2: Gradient descent, logistic MLE • TD3: Time series (AR/ARIMA/GARCH), OLS • TD4: VaR, backtesting, CoVaR • TD5: Markowitz optimization • TD6: Index construction

Session Map

Session	Topic	Notebook
TD 1	Python foundations, BSM Greeks, Monte Carlo	*Exercices - M1_eco_fi.ipynb*
TD 2	Gradient descent, logistic MLE	*Gradient Descent Students.ipynb*
TD 3	Time series: AR/ARIMA/GARCH, OLS	*Session 3 Time series modelling.ipynb*
TD 4	VaR, backtesting, CoVaR	*Session 4 Systemic risk ... .ipynb*
TD 5	Markowitz optimization	*TP Markowitz CAC 40.ipynb*
TD 6	Index construction	*Index_construction_introduction.ipynb*

Roadmap

1. TD2 – Gradient Descent & Logistic MLE

Outline

1. TD2 – Gradient Descent & Logistic MLE

Gradient Descent: Setting Up the Problem

Let $f : \mathbb{R}^p \to \mathbb{R}$ be a differentiable objective function we want to minimize, with parameter vector $\beta = (\beta_1, \ldots, \beta_p)^\top \in \mathbb{R}^p$.

$$\boxed{\beta^\star = \arg\min_{\beta \in \mathbb{R}^p} f(\beta)}$$

Gradient descent builds a sequence $\beta_0, \beta_1, \beta_2, \ldots$ that progressively reduces f.

Important caveat

Whether this sequence converges to a local minimum, a global minimum, or merely a stationary point depends on additional properties of f – this is exactly what we investigate below.

The Gradient: Direction of Steepest Change

The gradient of f at β collects all first-order partial derivatives:

$$\boxed{\nabla f(\beta) = \begin{pmatrix} \partial f(\beta)/\partial \beta_1 \\ \vdots \\ \partial f(\beta)/\partial \beta_p \end{pmatrix} \in \mathbb{R}^p}$$

- $\nabla f(\beta)$ points in the direction of **steepest local increase** of f (w.r.t. the Euclidean norm).
- Hence $\boxed{-\nabla f(\beta)}$ points in the direction of **steepest local decrease**.

This is the core idea of gradient descent: move *against* the gradient at every iteration.

Reminder: the Taylor Expansion

For $f : \mathbb{R} \to \mathbb{R}$ sufficiently differentiable, around a point a:

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \cdots = \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k + R_n(x)$$

Term	Information captured
$f(a)$	value of the function at a
$f'(a)(x-a)$	local slope
$\frac{1}{2}f''(a)(x-a)^2$	local curvature

Keeping only the first-order term (valid for x close to a) gives the **first-order approximation**: with $x = a + \Delta x$,

$$f(a + \Delta x) \approx f(a) + f'(a)\Delta x$$

The Multivariate Taylor Expansion

For $f : \mathbb{R}^p \to \mathbb{R}$ and a small displacement $\Delta\beta \in \mathbb{R}^p$:

$$\boxed{f(\beta + \Delta\beta) = f(\beta) + \nabla f(\beta)^\top \Delta\beta + \frac{1}{2}\Delta\beta^\top H_f(\beta)\Delta\beta + \cdots}$$

where $H_f(\beta) = \nabla^2 f(\beta)$ is the Hessian matrix (second-order partial derivatives). Both $\nabla f(\beta)^\top \Delta\beta$ and $\Delta\beta^\top H_f(\beta)\Delta\beta$ are scalars (check the dimensions: $1\times p$ times $p\times 1$, and $1\times p$ times $p\times p$ times $p\times 1$).
If $\Delta\beta$ is small, we neglect second- and higher-order terms:

$$\boxed{f(\beta + \Delta\beta) \approx f(\beta) + \nabla f(\beta)^\top \Delta\beta}$$

Applying Taylor's Expansion to a Gradient Step

Gradient descent chooses the displacement $\boxed{\Delta\beta = -\gamma \nabla f(\beta),\ \gamma > 0}$.

Substituting into the first-order approximation $f(\beta + \Delta\beta) \approx f(\beta) + \nabla f(\beta)^\top \Delta\beta$:

$$f\big(\beta - \gamma \nabla f(\beta)\big) \approx f(\beta) + \nabla f(\beta)^\top \big(-\gamma \nabla f(\beta)\big)$$

$$\boxed{f\big(\beta - \gamma \nabla f(\beta)\big) \approx f(\beta) - \gamma\, \nabla f(\beta)^\top \nabla f(\beta)}$$

We now need to interpret the quantity $\nabla f(\beta)^\top \nabla f(\beta)$.

The Euclidean Norm of the Gradient

For $x = (x_1, \ldots, x_p)^\top$, the Euclidean ($L^2$) norm is $\boxed{\|x\|_2 = \sqrt{\sum_{j=1}^{p} x_j^2}}$, so

$$\boxed{\|\nabla f(\beta)\|_2^2 = \sum_{j=1}^{p} \left(\frac{\partial f}{\partial \beta_j} \right)^2}$$

The inner product of a vector with itself equals the sum of squares of its components:

$$\nabla f(\beta)^\top \nabla f(\beta) = \left(\frac{\partial f}{\partial \beta_1} \right)^2 + \cdots + \left(\frac{\partial f}{\partial \beta_p} \right)^2$$

$$\boxed{\nabla f(\beta)^\top \nabla f(\beta) = \|\nabla f(\beta)\|_2^2}$$

$\|\nabla f(\beta)\|_2$ is the length/magnitude of the gradient: how strongly f changes locally around β.

Conclusion: the Negative Gradient Locally Decreases f

Combining $f(\beta - \gamma \nabla f(\beta)) \approx f(\beta) - \gamma \, \nabla f(\beta)^\top \nabla f(\beta)$ with $\nabla f(\beta)^\top \nabla f(\beta) = \|\nabla f(\beta)\|_2^2$:

$$\boxed{f\big(\beta - \gamma \nabla f(\beta)\big) \approx f(\beta) - \gamma \|\nabla f(\beta)\|_2^2}$$

Since $\gamma > 0$ and $\|\nabla f(\beta)\|_2^2 \geq 0$, the second term is non-positive, so for a sufficiently small step:

$$\boxed{f\big(\beta - \gamma \nabla f(\beta)\big) \lesssim f(\beta)}$$

This is the **local** justification for moving against the gradient.

But this is only an approximation

The Taylor relation above does not by itself *prove* that every gradient-descent step decreases f. A rigorous guarantee requires an extra smoothness assumption: the gradient must be **Lipschitz continuous** (see next).

From Local Intuition to a Rigorous Descent Guarantee

If ∇f is L-Lipschitz, the **descent lemma** (derived later) gives the exact inequality

$$\boxed{f(\beta - \gamma \nabla f(\beta)) \leq f(\beta) - \gamma\left(1 - \frac{L\gamma}{2}\right)\|\nabla f(\beta)\|_2^2}$$

$$\boxed{\text{Taylor expansion} \implies \text{local intuition}}$$

$$\boxed{L\text{-Lipschitz gradient + step size} \implies \text{rigorous guarantee}}$$

The Gradient Descent Update Rule

Starting from $\boxed{\beta_0 \in \mathbb{R}^p}$, at each iteration t compute $\nabla f(\beta_t)$ and update:

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t \nabla f(\beta_t)}$$

(dimension check: $\underbrace{\beta_{t+1}}_{p\times 1} = \underbrace{\beta_t}_{p\times 1} - \underbrace{\gamma_t}_{1\times 1}\underbrace{\nabla f(\beta_t)}_{p\times 1}$).

Three separate questions

1. Does each iteration decrease the objective? (needs: L-Lipschitz gradient + appropriate step size)
2. Do the gradients approach zero? (needs: additionally, f bounded below)
3. Does a stationary point correspond to a minimum? (needs: convexity)

Condition 1: Differentiability

$f : \mathbb{R}^p \to \mathbb{R}$ is **differentiable at** β if there is a linear map $\nabla f(\beta)$ such that, as $h \to 0$,

$$\boxed{f(\beta + h) = f(\beta) + \nabla f(\beta)^\top h + o(\|h\|_2)}$$

where $o(\|h\|_2)$ means the remainder $R(h)$ vanishes **faster** than $\|h\|_2$:

$$\boxed{\lim_{\|h\|_2 \to 0} \frac{f(\beta + h) - f(\beta) - \nabla f(\beta)^\top h}{\|h\|_2} = 0}$$

So, close to β, f is well approximated by an affine function: $f(\beta + h) \approx f(\beta) + \nabla f(\beta)^\top h$. Differentiability means f admits a first-order linear approximation at every point.

How to Show Differentiability in Practice

A convenient **sufficient** condition: show that every partial derivative $\partial f / \partial \beta_j$ **exists** and is **continuous** near the point of interest. Then

$$\boxed{f \in C^1 \implies f \text{ is differentiable}}$$

Existence of a partial derivative at β_0 means the limit

$$\boxed{\frac{\partial f(\beta_0)}{\partial \beta_j} = \lim_{h \to 0} \frac{f(\beta_0 + h e_j) - f(\beta_0)}{h}}$$

exists and is finite, where e_j is the j-th canonical basis vector (changes only coordinate j).

Careful

Existence of all partial derivatives alone does **not** generally imply differentiability – continuity of those partials is what provides the sufficient condition.

Continuity of the Partial Derivatives

Write $g_j(\beta) = \partial f(\beta)/\partial \beta_j$. We need g_j **continuous**:

$$\boxed{\lim_{\beta \to \beta_0} \frac{\partial f(\beta)}{\partial \beta_j} = \frac{\partial f(\beta_0)}{\partial \beta_j}} \qquad \forall j = 1, \ldots, p$$

If this holds everywhere on the domain, ∇f is continuous and we write $\boxed{f \in C^1}$, hence f is differentiable.

Do not confuse continuity and differentiability

$\boxed{f \text{ differentiable} \implies f \text{ continuous}}$ but $\boxed{f \text{ continuous} \not\implies f \text{ differentiable}}$

Example: $f(x) = |x|$ is continuous but not differentiable at $x = 0$.

Condition 2: Lipschitz-Continuous Gradient

A second condition controls how fast the gradient itself can change. There must exist $L > 0$ such that

$$\boxed{\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2} \qquad \text{for all relevant } x, y$$

A function satisfying this is called **L-smooth**: the slope of f cannot vary arbitrarily fast, and L bounds how rapidly it can.

Method 1 (direct): compute $\nabla f(x) - \nabla f(y)$ and find a finite constant L that bounds the ratio for *every* x, y in the domain.

Verifying the Lipschitz Property via the Hessian

Method 2: for a twice-differentiable f, study the Hessian $\nabla^2 f(\beta)$. If

$$\boxed{\|\nabla^2 f(\beta)\|_2 \leq L} \qquad \text{for every } \beta \text{ in the domain}$$

then ∇f is L-Lipschitz. In practice:

1. Compute $\nabla f(\beta)$ and $\nabla^2 f(\beta)$.
2. Find an upper bound L on the spectral norm $\|\nabla^2 f(\beta)\|_2$.
3. Verify the same finite L works throughout the relevant domain.

$$\boxed{\sup_{\beta} \|\nabla^2 f(\beta)\|_2 < \infty \implies \nabla f \text{ is Lipschitz continuous}}$$

A Hessian may be bounded on a restricted domain even if unbounded over all of $\mathbb{R}^p$.

The Appropriate Learning Rate

The learning rate γ is a choice made by the *algorithm*, not a property of f. Suppose ∇f is L-Lipschitz. For a constant rate $\gamma_t = \gamma$, the standard condition is

$$\boxed{0 < \gamma < \frac{2}{L}}$$

with the common, more conservative choice

$$\boxed{0 < \gamma \leq \frac{1}{L}}$$

The Descent Lemma

If ∇f is L-Lipschitz, then for any x, y:

$$\boxed{f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2}\|y - x\|_2^2}$$

For gradient descent, $y = x - \gamma \nabla f(x)$, so $y - x = -\gamma \nabla f(x)$. Substituting:

$$f(x - \gamma \nabla f(x)) \leq f(x) - \gamma \, \nabla f(x)^\top \nabla f(x) + \frac{L}{2}\gamma^2 \|\nabla f(x)\|_2^2$$

Using $\nabla f(x)^\top \nabla f(x) = \|\nabla f(x)\|_2^2$ and factoring:

$$\boxed{f(x - \gamma \nabla f(x)) \leq f(x) - \gamma\left(1 - \frac{L\gamma}{2}\right)\|\nabla f(x)\|_2^2}$$

The Descent Lemma Guarantees Strict Decrease

At iteration k:

$$\boxed{f(\beta_{k+1}) \leq f(\beta_k) - \gamma\left(1 - \frac{L\gamma}{2}\right)\|\nabla f(\beta_k)\|_2^2}$$

If $0 < \gamma < 2/L$, then $1 - L\gamma/2 > 0$. Consequently, whenever $\nabla f(\beta_k) \neq 0$:

$$\boxed{f(\beta_{k+1}) < f(\beta_k)}$$

Conclusion

Each gradient-descent step **strictly decreases** the objective whenever the gradient is non-zero – this is the rigorous version of the Taylor-based intuition.

Boundedness Below $\Rightarrow$ Gradients Vanish

Decrease alone does not imply $\nabla f \to 0$. Add the standard assumption: f is **bounded below**, $\boxed{f(\beta) \geq f_{\inf} > -\infty \quad \forall \beta}$.
Let $c = \gamma(1 - L\gamma/2) > 0$ (true for $0 < \gamma < 2/L$). The descent inequality becomes $f(\beta_{k+1}) \leq f(\beta_k) - c\|\nabla f(\beta_k)\|_2^2$, i.e.

$$\boxed{c\|\nabla f(\beta_k)\|_2^2 \leq f(\beta_k) - f(\beta_{k+1})}$$

Summing $k = 0, \ldots, K-1$, the right-hand side **telescopes**:
$\sum_{k=0}^{K-1} [f(\beta_k) - f(\beta_{k+1})] = f(\beta_0) - f(\beta_K)$.

Boundedness Below $\Rightarrow$ Gradients Vanish (cont'd)

So $c \sum_{k=0}^{K-1} \|\nabla f(\beta_k)\|_2^2 \leq f(\beta_0) - f(\beta_K)$. Since $f(\beta_K) \geq f_{\text{inf}}$:

$$c \sum_{k=0}^{K-1} \|\nabla f(\beta_k)\|_2^2 \leq f(\beta_0) - f_{\text{inf}}$$

Letting $K \to \infty$:

$$\boxed{\sum_{k=0}^{\infty} \|\nabla f(\beta_k)\|_2^2 < \infty}$$

A convergent series of non-negative terms has terms $\to 0$, hence

$$\boxed{\|\nabla f(\beta_k)\|_2 \longrightarrow 0}$$

This is the standard **stationarity guarantee** for smooth gradient descent.

Stationary Points

$\beta^\star$ is a **stationary point** if $\boxed{\nabla f(\beta^\star) = 0}$. Under

$$\boxed{\nabla f \text{ is } L\text{-Lipschitz} \;+\; 0 < \gamma < 2/L \;+\; f \text{ bounded below}}$$

we obtain $\boxed{\|\nabla f(\beta_k)\|_2 \to 0}$: gradient descent approaches stationarity.

Gradient convergence $\neq$ parameter convergence

$\|\nabla f(\beta_k)\|_2 \to 0$ and $\beta_k \to \beta^\star$ are **different** statements. In general non-convex problems, a vanishing gradient norm does not by itself imply the whole sequence $\{\beta_k\}$ converges to one point – extra assumptions are needed.

When Is a Stationary Point a Minimum? Convexity

A stationary point ($\nabla f(\beta^\star) = 0$) may be a local min, a local max, or a saddle point in general. **Convexity** changes this.

f is convex if $\boxed{f(y) \geq f(x) + \nabla f(x)^\top (y - x)}$ for all x, y (the graph lies above every tangent hyperplane).

Apply this with $x = \beta^\star$ stationary: $f(y) \geq f(\beta^\star) + \nabla f(\beta^\star)^\top (y - \beta^\star)$. Since $\nabla f(\beta^\star) = 0$:

$$f(y) \geq f(\beta^\star) \qquad \forall y$$

$$\boxed{\nabla f(\beta^\star) = 0 \;+\; f \text{ convex} \implies \beta^\star \in \arg\min_{\beta} f(\beta)}$$

$$\boxed{\text{stationary point} \implies \text{global minimum} \qquad \text{(convex case)}}$$

Strong Convexity

f is μ-**strongly convex** if there is $\mu > 0$ with

$$\boxed{f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2}\|y - x\|_2^2} \qquad \forall x, y$$

The extra quadratic term forces a minimum amount of curvature.

$\boxed{\text{strong convexity} \implies \text{convexity}}$ $\boxed{\text{strong convexity} \implies \text{unique global minimizer}}$

Rate of convergence

If f is both L-smooth **and** μ-strongly convex, gradient descent with an appropriate constant step converges **linearly (geometrically)** to the unique minimizer: the error shrinks by a constant factor every iteration, not just asymptotically.

The Non-Convex Case

If f is non-convex, several stationary points can exist (local minima, local maxima, saddle points), and gradient descent is **not** generally guaranteed to reach a global minimum – the outcome can depend on the initialization β_0.

$$\boxed{\text{non-convexity} \implies \text{stationarity does not imply global optimality}}$$

This matters a great deal in machine learning, where many objectives are non-convex.

Summary of the Convergence Hierarchy

f differentiable + ∇f L-Lipschitz + $0 < \gamma < 2/L$

$\Downarrow$

$f(\beta_{k+1}) \leq f(\beta_k)$ (descent)

descent + f bounded below

$\Downarrow$

$\|\nabla f(\beta_k)\|_2 \to 0$ (stationarity)

stationarity + f convex $\implies$ global minimum

$+$ strong convexity $\implies$ unique + linear convergence

What Each Assumption Buys You

Assumption	**Main role**
Differentiability	Makes the gradient well defined
L-Lipschitz gradient	Controls how rapidly the gradient changes
$0 < \gamma < 2/L$	Appropriate gradient-descent step size
f bounded below	Prevents indefinite descent toward $-\infty$
Convexity	Makes stationary points global minima
Strong convexity	Unique minimum and stronger (linear) convergence

Choosing the Step Size γ_t

A **constant** rate $\gamma_t = \gamma$ with $0 < \gamma < 2/L$ (conservatively $\gamma \leq 1/L$) is one option.
Alternatively, a **decreasing** rate:

$$\boxed{\gamma_t = \frac{c}{\sqrt{t}}} \qquad t \geq 1,\ c > 0$$

Early iterations take larger steps; later iterations refine the estimate with smaller steps.

Step size	Possible consequence
Too small	very slow progress
Appropriate	stable descent / convergence
Too large	oscillation or divergence

Stopping Criterion

At a differentiable local minimum, $\nabla f(\beta^\star) = 0$ is **necessary** but, without convexity, not sufficient to conclude it is a minimum. Numerically the gradient is never exactly zero, so fix a tolerance $\varepsilon > 0$ (e.g. 10^{-8}) and stop when

$$\boxed{\|\nabla f(\beta_t)\|_2 < \varepsilon}$$

The algorithm also stops once a maximum number of iterations $K_{\max}$ is reached.

The Complete Gradient Descent Algorithm

Given $f : \mathbb{R}^p \to \mathbb{R}$, $\beta_0 \in \mathbb{R}^p$, $\varepsilon > 0$, $K_{\max} \in \mathbb{N}$, repeat for $t = 0, \ldots, K_{\max} - 1$:

1. Compute $g_t = \nabla f(\beta_t)$.
2. If $\|g_t\|_2 < \varepsilon$: **stop**, return β_t.
3. Otherwise choose $\gamma_t > 0$ and update $\boxed{\beta_{t+1} = \beta_t - \gamma_t g_t}$.
4. Repeat.

The One-Dimensional Case

With $f : \mathbb{R} \to \mathbb{R}$ and $\beta \in \mathbb{R}$, the gradient reduces to the ordinary derivative: $\boxed{\nabla f(\beta) = f'(\beta)}$, so

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t f'(\beta_t)}$$

- If $f'(\beta_t) > 0$ (locally increasing): $\beta_{t+1} = \beta_t - \gamma_t f'(\beta_t) < \beta_t$ – the algorithm moves **left**.
- If $f'(\beta_t) < 0$: $-\gamma_t f'(\beta_t) > 0$, so $\beta_{t+1} > \beta_t$ – the algorithm moves **right**.
- Near a stationary point, $f'(\beta_t) \approx 0$, so $\beta_{t+1} \approx \beta_t$.

Local versus Global Minimum

Gradient descent uses only **local, first-order information**: the update at t depends only on $\nabla f(\beta_t)$.

- If f convex: every local minimum is global, every stationary point is global – gradient descent converges to a global minimizer $\beta^\star \in \arg\min_\beta f(\beta)$.
- If f non-convex: multiple local minima/stationary points may exist; the outcome can depend on β_0.

Key idea

Gradient descent seeks a **stationary** solution; without convexity, this need not be the global minimum.

Key Idea: Gradient Descent in Three Operations

Evaluate slope	$\longrightarrow$	Move downhill
$\nabla f(\beta_t)$		$-\nabla f(\beta_t)$
		$\downarrow$
		$\beta_{t+1} = \beta_t - \gamma_t \nabla f(\beta_t)$

The key conceptual distinction

Smoothness $\rightarrow$ controls the *behavior*

Convexity $\rightarrow$ determines what stationarity *implies*

Case Study: Logistic Regression by Maximum Likelihood

Logistic regression models a binary dependent variable (a classification model in ML language). **Goal:** recover the parameter $\beta^\star$ used to generate binary observations $Y_1, \ldots, Y_n$, by deriving in turn:

1. the probability model implied by a latent-variable specification;
2. the likelihood, then the log-likelihood, of the observations;
3. the negative log-likelihood and its gradient;
4. the gradient-descent update used to estimate $\beta^\star$.

The Latent-Variable Representation

For $i = 1, \ldots, n$, introduce the (unobserved) latent variable

$$\boxed{Y_i^\star = \beta x_i + \varepsilon_i}, \qquad \beta, x_i, \varepsilon_i \in \mathbb{R}$$

We observe only the binary indicator

$$\boxed{Y_i = \begin{cases} 1, & Y_i^\star > 0 \\ 0, & \text{otherwise} \end{cases}}$$

Substituting: $Y_i = 1 \iff \beta x_i + \varepsilon_i > 0$, i.e.

$$\boxed{Y_i = 1 \quad \iff \quad \varepsilon_i > -\beta x_i}$$

The Logistic Error Density and Its CDF

ε_i follows the standard logistic distribution with density

$$\boxed{f_\varepsilon(z) = \frac{e^{-z}}{(1+e^{-z})^2}}, \qquad z \in \mathbb{R}$$

By definition, its CDF is the integral of the density:

$$\boxed{F_\varepsilon(z) = \mathbb{P}(\varepsilon_i \le z) = \int_{-\infty}^{z} \frac{e^{-u}}{(1+e^{-u})^2}\, du}$$

We now compute this integral explicitly via a change of variable.

Step 1: Change of Variable

Define $\boxed{v = 1 + e^{-u}}$. Then $dv/du = -e^{-u}$, so

$$\boxed{e^{-u}\, du = -dv}$$

Transforming the bounds:

u	$v = 1 + e^{-u}$
$-\infty$	$+\infty$
z	$1 + e^{-z}$

(as $u \to -\infty$, $-u \to +\infty$, so $e^{-u} \to +\infty$, hence $v \to +\infty$). The integral becomes

$$\boxed{F_\varepsilon(z) = -\int_{+\infty}^{1+e^{-z}} v^{-2}\, dv}$$

Step 2: Integrate

Power rule: $\boxed{\int v^a \, dv = \frac{v^{a+1}}{a+1} + C}$ $(a \neq -1)$. With $a = -2$: $a + 1 = -1$, so

$$\boxed{\int v^{-2} \, dv = -v^{-1} + C = -\frac{1}{v} + C}$$

Check by differentiating: $\frac{d}{dv}\left(-v^{-1}\right) = -(-1)v^{-2} = v^{-2}$. ✓

So $F_\varepsilon(z) = -\left[-\frac{1}{v}\right]_{+\infty}^{1+e^{-z}}$; the two minus signs cancel:

$$\boxed{F_\varepsilon(z) = \left[\frac{1}{v}\right]_{+\infty}^{1+e^{-z}}}$$

Step 3: Evaluate at the Bounds

Using $\int_a^b g(v)\,dv = [G(v)]_a^b = G(b) - G(a)$:

$$F_\varepsilon(z) = \frac{1}{1+e^{-z}} - \lim_{v \to +\infty} \frac{1}{v}$$

Since $\boxed{\lim_{v \to +\infty} 1/v = 0}$:

$$\boxed{F_\varepsilon(z) = \frac{1}{1+e^{-z}}}$$

So the logistic CDF is $\boxed{F_\varepsilon(z) = \mathbb{P}(\varepsilon_i \leq z) = \frac{1}{1+e^{-z}}}$. This is exactly the **logistic (sigmoid) function** $\boxed{\sigma(z) = \frac{1}{1+e^{-z}}}$, so $F_\varepsilon = \sigma$.

Verification: Differentiating F_ε Should Recover f_ε

A valid CDF must satisfy $f_\varepsilon(z) = \dfrac{d}{dz} F_\varepsilon(z)$. Write $F_\varepsilon(z) = (1 + e^{-z})^{-1} = g(u(z))$ with outer/inner functions

$$g(u) = u^{-1}, \qquad u(z) = 1 + e^{-z}$$

Reminder (chain rule): for $h(z) = g(u(z))$, $\boxed{h'(z) = g'(u(z))\, u'(z)}$ – differentiate the outer function first, multiply by the derivative of the inner function.

Step 1–2: Differentiate the Outer and Inner Functions

Outer: $g(u) = u^{-1}$. Power rule $\frac{d}{du}u^a = au^{a-1}$ with $a = -1$ gives $g'(u) = -u^{-2}$, so

$$\boxed{g'(u(z)) = -(1 + e^{-z})^{-2}}$$

Inner: $u(z) = 1 + e^{-z}$. The derivative of a constant is 0, and $\frac{d}{dz}e^{-z} = -e^{-z}$, so

$$\boxed{u'(z) = -e^{-z}}$$

Step 3–4: Combine via the Chain Rule

$$F'_\varepsilon(z) = g'(u(z))\,u'(z) = \big[-(1+e^{-z})^{-2}\big]\big[-e^{-z}\big]$$

The two minus signs cancel:

$$\boxed{F'_\varepsilon(z) = e^{-z}(1+e^{-z})^{-2} = \frac{e^{-z}}{(1+e^{-z})^2}}$$

This is exactly $f_\varepsilon(z)$:

$$\boxed{F'_\varepsilon(z) = f_\varepsilon(z)} \quad \checkmark$$

Differentiating the derived CDF correctly recovers the logistic density – the derivation is self-consistent.

Symmetry of the Logistic Distribution

The standard logistic distribution is symmetric about 0. From $F_\varepsilon(-z) = \dfrac{1}{1+e^z}$:

$$1 - F_\varepsilon(-z) = 1 - \frac{1}{1+e^z} = \frac{1+e^z-1}{1+e^z} = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$$

(dividing numerator and denominator by e^z). But this equals $F_\varepsilon(z)$, so

$$\boxed{F_\varepsilon(-z) = 1 - F_\varepsilon(z)}$$

This symmetry is exactly what we need to derive $\mathbb{P}(Y_i = 1 \mid x_i)$ next.

Explanatory Variables

For $i = 1, \ldots, n$: $\boxed{x_i \overset{iid}{\sim} \mathcal{N}(0, 1)}$.

The x_i are i.i.d., but conditional on x_i the distribution of Y_i **depends** on x_i: $\mathbb{P}(Y_i = 1 \mid x_i)$ varies with x_i. So the Y_i are independent but, conditional on the regressors, **not identically distributed**.

Deriving the Conditional Probability $\mathbb{P}(Y_i = 1 \mid x_i)$

From the latent model, $Y_i = 1 \iff Y_i^\star > 0 \iff \beta x_i + \varepsilon_i > 0 \iff \varepsilon_i > -\beta x_i$. Hence

$$\mathbb{P}(Y_i = 1 \mid x_i) = \mathbb{P}(\varepsilon_i > -\beta x_i) = 1 - F_\varepsilon(-\beta x_i)$$

Using the symmetry $1 - F_\varepsilon(-z) = F_\varepsilon(z)$ with $z = \beta x_i$:

$$\mathbb{P}(Y_i = 1 \mid x_i) = F_\varepsilon(\beta x_i) = \frac{1}{1 + e^{-\beta x_i}}$$

$$\boxed{p_i(\beta) = \sigma(\beta x_i) = \frac{1}{1 + e^{-\beta x_i}}} \qquad \boxed{Y_i \mid x_i \sim \text{Bernoulli}(p_i(\beta))}$$

The Complementary Probability

By the same logic, $\mathbb{P}(Y_i = 0 \mid x_i) = 1 - p_i(\beta)$. Since

$$1 - \frac{1}{1 + e^{-\beta x_i}} = \frac{e^{-\beta x_i}}{1 + e^{-\beta x_i}}$$

we get

$$\boxed{\mathbb{P}(Y_i = 0 \mid x_i) = \frac{e^{-\beta x_i}}{1 + e^{-\beta x_i}} = \frac{1}{1 + e^{\beta x_i}}}$$

A Single Formula for Both Outcomes

Since $Y_i \in \{0, 1\}$, the probability of the realized outcome y_i is written compactly as

$$\boxed{\mathbb{P}(Y_i = y_i \mid x_i; \beta) = p_i(\beta)^{y_i} \big(1 - p_i(\beta)\big)^{1-y_i}}$$

Check both cases:

- $y_i = 1$: $p_i^1(1 - p_i)^0 = p_i$ ✓
- $y_i = 0$: $p_i^0(1 - p_i)^1 = 1 - p_i$ ✓

One formula covers both possible outcomes – this is what makes the likelihood below easy to write.

The Likelihood Function

Conditional on $x_1, \ldots, x_n$, the observations are independent, so the joint probability of the full sample is the product of individual probabilities:

$$L(\beta) = \prod_{i=1}^{n} \mathbb{P}(Y_i = y_i \mid x_i; \beta) = \prod_{i=1}^{n} p_i(\beta)^{y_i} \big(1 - p_i(\beta)\big)^{1-y_i}$$

L is viewed as a function of the unknown β, with the data (x_i, y_i) fixed. The **maximum-likelihood estimator** is

$$\hat{\beta}_{MLE} = \arg\max_{\beta \in \mathbb{R}} L(\beta)$$

Taking Logarithms

Products are hard to differentiate and numerically unstable (can underflow). Since log is strictly increasing, $\arg\max_\beta L(\beta) = \arg\max_\beta \log L(\beta)$. Define

$$\boxed{\ell(\beta) = \log L(\beta)}$$

Using $\log(\prod_i a_i) = \sum_i \log(a_i)$:

$$\ell(\beta) = \sum_{i=1}^{n} \log \left[p_i^{y_i} (1 - p_i)^{1-y_i} \right]$$

Expanding the Log-Likelihood

Using $\log(ab) = \log a + \log b$:

$$\ell(\beta) = \sum_{i=1}^{n} \left[\log(p_i^{y_i}) + \log\left((1-p_i)^{1-y_i}\right) \right]$$

Using $\log(a^b) = b \log a$ on each term:

$$\log(p_i^{y_i}) = y_i \log p_i, \qquad \log\left((1-p_i)^{1-y_i}\right) = (1-y_i)\log(1-p_i)$$

$$\boxed{\ell(\beta) = \sum_{i=1}^{n} \left[y_i \log p_i + (1-y_i)\log(1-p_i) \right]}$$

This is the familiar Bernoulli log-likelihood.

Substituting the Logistic Probabilities

Recall $p_i = \dfrac{1}{1+e^{-\beta x_i}}$, so $\log p_i = -\log(1+e^{-\beta x_i})$.

Also $1 - p_i = \dfrac{e^{-\beta x_i}}{1+e^{-\beta x_i}}$, so using $\log(a/b) = \log a - \log b$:

$$\boxed{\log(1-p_i) = -\beta x_i - \log(1+e^{-\beta x_i})}$$

Substitute both into $\ell(\beta) = \sum_i [y_i \log p_i + (1-y_i)\log(1-p_i)]$:

$$\ell(\beta) = \sum_{i=1}^{n} \Big[-y_i \log(1+e^{-\beta x_i}) + (1-y_i)\big(-\beta x_i - \log(1+e^{-\beta x_i})\big) \Big]$$

Collecting Terms

Expanding:

$$\ell(\beta) = \sum_{i=1}^{n} \Big[- y_i \log(1 + e^{-\beta x_i}) - (1 - y_i)\beta x_i - (1 - y_i) \log(1 + e^{-\beta x_i}) \Big]$$

The two logarithmic terms combine as $-[y_i + (1 - y_i)] \log(1 + e^{-\beta x_i})$, and since $y_i + (1 - y_i) = 1$, this is simply $-\log(1 + e^{-\beta x_i})$. So

$$\boxed{\ell(\beta) = -\sum_{i=1}^{n} \log(1 + e^{-\beta x_i}) - \beta \sum_{i=1}^{n} (1 - y_i) x_i}$$

The Final Simplified Log-Likelihood

Because $y_i \in \{0, 1\}$, $1 - y_i$ equals 1 exactly when $y_i = 0$, so $\sum_{i=1}^{n}(1 - y_i)x_i = \sum_{i:y_i=0} x_i$. Hence

$$\boxed{\ell(\beta) = -\sum_{i=1}^{n} \log(1 + e^{-\beta x_i}) - \beta \sum_{i:y_i=0} x_i}$$

and the maximum-likelihood problem is

$$\boxed{\max_{\beta \in \mathbb{R}} \left[-\sum_{i=1}^{n} \log(1 + e^{-\beta x_i}) - \beta \sum_{i:y_i=0} x_i \right]}$$

From Maximization to Minimization

Gradient descent is conventionally a **minimization** algorithm, but MLE requires $\max_\beta \ell(\beta)$. Flip the sign:

$$\boxed{\arg\max_\beta \ell(\beta) = \arg\min_\beta [-\ell(\beta)]}$$

Define the negative log-likelihood $\boxed{J(\beta) = -\ell(\beta)}$, i.e.

$$\boxed{J(\beta) = \beta \sum_{i:y_i=0} x_i + \sum_{i=1}^{n} \log(1 + e^{-\beta x_i})}$$

so the estimation problem becomes $\boxed{\hat{\beta}_{MLE} = \arg\min_{\beta \in \mathbb{R}} J(\beta)}$.

Differentiating $J(\beta)$ Term by Term

With a single parameter $\beta \in \mathbb{R}$, $\nabla J(\beta) = J'(\beta)$. Recall

$$J(\beta) = \beta \sum_{i:y_i=0} x_i + \sum_{i=1}^{n} \log(1 + e^{-\beta x_i})$$

First term: the x_i do not depend on β, so

$$\boxed{\frac{d}{d\beta}\left[\beta \sum_{i:y_i=0} x_i\right] = \sum_{i:y_i=0} x_i}$$

Differentiating the Second Term

For one element $g_i(\beta) = \log(1 + e^{-\beta x_i})$, set $u(\beta) = 1 + e^{-\beta x_i}$ so $g_i(\beta) = \log u(\beta)$. Since $\frac{d}{du} \log u = 1/u$:

$$g_i'(\beta) = \frac{1}{1 + e^{-\beta x_i}} \cdot \frac{d}{d\beta}(1 + e^{-\beta x_i})$$

Now $\frac{d}{d\beta} 1 = 0$ and $\frac{d}{d\beta} e^{-\beta x_i} = e^{-\beta x_i} \cdot \frac{d}{d\beta}(-\beta x_i) = -x_i e^{-\beta x_i}$. So

$$\boxed{g_i'(\beta) = -\frac{x_i e^{-\beta x_i}}{1 + e^{-\beta x_i}}} \qquad \Longrightarrow \qquad \frac{d}{d\beta} \sum_{i=1}^{n} \log(1 + e^{-\beta x_i}) = -\sum_{i=1}^{n} \frac{x_i e^{-\beta x_i}}{1 + e^{-\beta x_i}}$$

The Complete Gradient

Combining both derivatives:

$$\boxed{J'(\beta) = \sum_{i:y_i=0} x_i - \sum_{i=1}^{n} \frac{x_i e^{-\beta x_i}}{1 + e^{-\beta x_i}}}$$

Equivalently, since $J = -\ell$:

$$\boxed{-\ell'(\beta) = \sum_{i:y_i=0} x_i - \sum_{i=1}^{n} \frac{x_i e^{-\beta x_i}}{1 + e^{-\beta x_i}}}$$

This is the exact expression we will plug into the gradient-descent update.

Rewriting the Gradient Using p_i

Since $p_i(\beta) = \dfrac{1}{1+e^{-\beta x_i}}$, we have $1 - p_i(\beta) = \dfrac{e^{-\beta x_i}}{1+e^{-\beta x_i}}$, hence $\dfrac{x_i e^{-\beta x_i}}{1+e^{-\beta x_i}} = x_i(1-p_i)$. So

$$J'(\beta) = \sum_{i:y_i=0} x_i - \sum_{i=1}^{n} x_i(1-p_i) = \sum_{i=1}^{n}(1-y_i)x_i - \sum_{i=1}^{n}(1-p_i)x_i$$

(using $\sum_{i:y_i=0} x_i = \sum_i (1-y_i)x_i$). Combining the sums:

$$J'(\beta) = \sum_{i=1}^{n}\big[(1-y_i) - (1-p_i)\big]x_i = \sum_{i=1}^{n}(p_i - y_i)x_i$$

The Standard Logistic-Regression Gradient

$$\boxed{J'(\beta) = \sum_{i=1}^{n} x_i \big(p_i(\beta) - y_i\big)} \quad \Longleftrightarrow \quad \boxed{J'(\beta) = \sum_{i=1}^{n} x_i \big[\sigma(\beta x_i) - y_i\big]}$$

Equivalently, the gradient of the log-likelihood itself is

$$\boxed{\ell'(\beta) = \sum_{i=1}^{n} x_i \big[y_i - \sigma(\beta x_i)\big]}$$

This compact form – *"regressor times prediction error"* – is the one implemented directly in gradient-descent code.

Interpreting the Gradient: Prediction Error

$J'(\beta) = \sum_i x_i(p_i - y_i)$, where $p_i - y_i$ measures how wrong the model's predicted probability is relative to the observed outcome.

- $y_i = 1$, $p_i = 0.2$: $p_i - y_i = -0.8$ – model assigns a **low** probability to something that **did** happen.
- $y_i = 0$, $p_i = 0.9$: $p_i - y_i = 0.9$ – model assigns a **high** probability to something that did **not** happen.

$$J'(\beta) = \sum_{i=1}^{n} \underbrace{x_i}_{\text{exposure}} \underbrace{(p_i - y_i)}_{\text{prediction error}}$$

The Second Derivative: Is J Convex?

Since x_i, y_i don't depend on β: $J''(\beta) = \sum_i x_i \, \partial p_i / \partial \beta$. The derivative of the logistic function is

$$\boxed{\sigma'(z) = \sigma(z)\big[1 - \sigma(z)\big]}$$

so by the chain rule $\partial p_i / \partial \beta = x_i \, p_i(1 - p_i)$, giving

$$\boxed{J''(\beta) = \sum_{i=1}^{n} x_i^2 \, p_i(1 - p_i)}$$

Since $x_i^2 \geq 0$ and $0 < p_i < 1 \Rightarrow p_i(1 - p_i) > 0$:

$$\boxed{J''(\beta) \geq 0}$$

Convexity $\Rightarrow$ No Spurious Local Minima

J is **convex** in β (equivalently, ℓ is concave: $\ell''(\beta) \leq 0$). Hence any finite stationary point is automatically the global minimizer:

$$\boxed{J'(\hat{\beta}) = 0 \implies \hat{\beta} \in \arg\min_{\beta} J(\beta)}$$

This is exactly why gradient descent is well-suited to this problem: there is no risk of converging to a spurious local minimum.

A Lipschitz Constant for J'

For every probability $p \in [0,1]$, $p(1-p) = p - p^2$ is maximized at $p = \frac{1}{2}$, where $p(1-p) = \frac{1}{4}$. Hence

$$p(1-p) \leq \frac{1}{4} \qquad \Longrightarrow \qquad J''(\beta) = \sum_i x_i^2 p_i(1-p_i) \leq \frac{1}{4} \sum_{i=1}^{n} x_i^2$$

So a valid Lipschitz constant for J' is

$$\boxed{L = \frac{1}{4} \sum_{i=1}^{n} x_i^2} \qquad \Longrightarrow \qquad \boxed{|J'(x) - J'(y)| \leq L|x - y|}$$

This plugs directly into the step-size condition $0 < \gamma < 2/L$ derived earlier for general gradient descent.

The Gradient-Descent Update for $\hat{\beta}_{MLE}$

We solve $\hat{\beta} = \arg\min_\beta J(\beta)$ with the generic update $\beta_{t+1} = \beta_t - \gamma_t J'(\beta_t)$. Substituting the derived gradient:

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t \left[\sum_{i:y_i=0} x_i - \sum_{i=1}^{n} \frac{x_i e^{-\beta_t x_i}}{1 + e^{-\beta_t x_i}} \right]}$$

Writing $p_{i,t} = \sigma(\beta_t x_i)$, this simplifies to

$$\boxed{\beta_{t+1} = \beta_t - \gamma_t \sum_{i=1}^{n} x_i \left(p_{i,t} - y_i \right)}$$

Step-by-Step Estimation Procedure

Choose $\beta_0 \in \mathbb{R}$, $\varepsilon > 0$, $K_{\max} \in \mathbb{N}$. For $t = 0, \ldots, K_{\max} - 1$:

1. **Linear score:** $z_{i,t} = \beta_t x_i$.
2. **Probability:** $p_{i,t} = \sigma(z_{i,t}) = \dfrac{1}{1 + e^{-\beta_t x_i}}$.
3. **Gradient:** $\boxed{g_t = J'(\beta_t) = \sum_{i=1}^{n} x_i (p_{i,t} - y_i)}$.
4. **Stop** if $\boxed{|g_t| < \varepsilon}$, returning $\hat{\beta} = \beta_t$.
5. **Step size:** choose γ_t with $0 < \gamma < 2/L$, $L = \frac{1}{4} \sum_i x_i^2$ (conservatively $\gamma \leq 1/L$).
6. **Update:** $\boxed{\beta_{t+1} = \beta_t - \gamma_t g_t}$.

The Complete Derivation at a Glance

$$Y_i^\star = \beta x_i + \varepsilon_i, \quad Y_i = \mathbb{1}_{\{Y_i^\star > 0\}} \quad \Longrightarrow \quad \boxed{p_i(\beta) = \sigma(\beta x_i)}, \quad Y_i \mid x_i \sim \text{Bernoulli}(p_i)$$

$$\boxed{L(\beta) = \prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{1-y_i}} \;\rightarrow\; \boxed{\ell(\beta) = \sum_i \left[y_i \log p_i + (1-y_i)\log(1-p_i)\right]}$$

$$\rightarrow \quad \boxed{\ell(\beta) = -\sum_i \log(1+e^{-\beta x_i}) - \beta \sum_{i:y_i=0} x_i} \quad \rightarrow \quad J = -\ell$$

The Complete Derivation at a Glance (cont'd)

$$J'(\beta) = \sum_{i=1}^{n} x_i \left[\sigma(\beta x_i) - y_i\right] \quad \rightarrow \quad \beta_{t+1} = \beta_t - \gamma_t \sum_{i=1}^{n} x_i \left[\sigma(\beta_t x_i) - y_i\right]$$

$$J''(\beta) = \sum_{i=1}^{n} x_i^2 p_i(1 - p_i) \geq 0 \quad \Longrightarrow \quad J \text{ convex} \quad \Longrightarrow \quad J'(\hat{\beta}) = 0 \;\Rightarrow\; \hat{\beta} \text{ global minimizer}$$

Key Ideas: Statistics Meets Optimization

Latent-variable model $\Downarrow$ Logistic probability $p_i = \sigma(\beta x_i)$
$\Downarrow$ Bernoulli likelihood $L(\beta)$ $\Downarrow$ Log-likelihood $\ell(\beta)$
$\Downarrow$ Negative log-likelihood $J(\beta) = -\ell(\beta)$
$\Downarrow$ Gradient $J'(\beta) = \sum_i x_i(p_i - y_i)$
$\Downarrow$ Gradient descent $\beta_{t+1} = \beta_t - \gamma_t J'(\beta_t)$ $\Downarrow$ $\hat{\beta}_{MLE}$

Maximum Likelihood Estimation $\iff$ Optimization of the log-likelihood

For logistic regression, maximizing $\ell(\beta)$ is equivalent to minimizing $-\ell(\beta)$, which gradient descent solves directly.